2026 INTERVIEW GUIDE

30 Fine-Tuning Interview Questions

with Answers

LoRA, QLoRA, DPO, distillation, and when to say no. Spoken answers with real code.

Naved Khan
Senior Gen-AI Engineer

+ Follow

READ THIS FIRST

"Should we fine-tune?" is a trap question.

The expected answer in 2026 is usually "not yet", and how you get there is the whole test. Interviewers are checking whether you reach for the most expensive tool first, or whether prompting and RAG have to fail before you touch the weights.

Naved Khan
Senior Gen-AI Engineer

+ Follow

HOW FINE-TUNING INTERVIEWS RUN IN 2026

5 areas. 30 questions.

1	When & Why	Q1–6
2	The Method Zoo	Q7–12
3	Data	Q13–18
4	Training & Evals	Q19–24
5	Strategy	Q25–30

The when-and-why decisions are where offers are won. Methods you can memorize; judgment you have to show.

Naved Khan
Senior Gen-AI Engineer

+ Follow

Q1 When is fine-tuning the right call?

- After the cheaper tools max out: **prompting for behavior, RAG for knowledge**. Fine-tuning earns its place in four spots: structured-output reliability, domain vocabulary, tone and refusal control, and **cost compression by distilling into a small model**.
- In plain words: fine-tuning changes **how** the model behaves, not **what it knows**. Most "should we fine-tune" problems are actually knowledge problems, and that's RAG.

Q2 What can fine-tuning NOT do?

- Reliably **add fresh facts**. Baked-in knowledge goes stale on day one, can't cite a source, and updating it means retraining.
- The line that scores: **"Fine-tune the interface, retrieve the content."** Tone, format, and tool habits into the weights; facts stay in the index.

Naved Khan
Senior Gen-AI Engineer

Q3 Full fine-tuning vs PEFT. Why did PEFT win?

- Full fine-tuning updates **every weight**: expensive, needs big hardware, and risks wiping general skills, which is called catastrophic forgetting.
- PEFT freezes the base model and trains **tiny add-on parameters**, usually under 1% of the total. Cheaper, safer, and the adapters are swappable like plugins.

Q4 Explain LoRA like I've never seen it.

- Instead of editing the huge weight matrices, LoRA learns a **small correction**: two thin matrices whose product approximates the change you'd want.
- The base stays frozen, so the original skills survive. At serving time you **merge the adapter or hot-swap it**: one base model, many personalities.

```
LoraConfig(r=16, lora_alpha=32, target_modules=
["q_proj","v_proj"])
```

Naved Khan
Senior Gen-AI Engineer

Q5 What does QLoRA add?

- It **quantizes the frozen base to 4-bit** and trains LoRA adapters on top. Memory drops so far that a 7–8B model fine-tunes on a **single GPU**.
- Rule of thumb I'd say out loud: QLoRA when GPU memory is the constraint, plain LoRA when you have room and want cleaner numerics.

```
load_in_4bit=True  # QLoRA: 4-bit base, trainable adapters
```

Q6 How much data does SFT need?

- Less than people think, if it's clean: **hundreds to a few thousand** high-quality examples beat millions of noisy ones.
- The examples must look **exactly like the final task**, same format, same chat template. Then dedupe, scrub PII, and keep eval questions out of the training set.

Naved Khan
Senior Gen-AI Engineer

Q7 SFT, RLHF, DPO. Walk me through the pipeline.

- **SFT** teaches the task: show input-output pairs, model imitates. **Preference tuning** comes after: teach the model which of two answers humans prefer.
- **RLHF** does that with a reward model plus reinforcement learning. **DPO** skips the reward model and learns directly from preference pairs in one supervised-style pass.

```
DPOTrainer(model, train_dataset=preference_pairs)
```

Q8 Why has DPO become the default over RLHF?

- RLHF needs a separate reward model, PPO training that's **notoriously unstable**, and heavy annotation. DPO gets most of the benefit with one stable training run.
- The senior framing: DPO is the **workhorse**, classic RLHF is the **escalation path** for frontier-scale alignment. Most product teams never need the escalation.

Naved Khan
Senior Gen-AI Engineer

Q9 What is distillation, and when do you reach for it?

- A big teacher model generates outputs, a small student model is fine-tuned to **imitate them**. You trade a little quality ceiling for 10× cheaper, faster serving.
- It's the endgame of the cost cascade: prove the pipeline with the big model, then **distill the routine traffic into a small one**. Check the teacher's license first; not all allow it.

Q10 Continued pretraining vs SFT. What's the difference?

- **Continued pretraining**: feed raw domain text so the model absorbs the vocabulary of law, medicine, or your codebase. No labels, just more reading.
- **SFT**: labeled input-output pairs teaching a specific task. Continued pretraining when the model can't even speak your domain's language; SFT when it speaks it but does the wrong thing.

Naved Khan
Senior Gen-AI Engineer

Q11 What is catastrophic forgetting, and how do you prevent it?

- Fine-tuning on narrow data can **overwrite general skills**: the model gets great at your task and quietly worse at everything else.
- Defenses: LoRA itself (the frozen base is protection), **low learning rates, few epochs**, mixing in general data, and evaluating on a general suite, not just your task.

Q12 Which hyperparameters actually matter?

- **Learning rate** first: too high destroys the model fast. **Epochs** second: LLMs overfit in 1–3 passes, not 50.
- Then LoRA's r and alpha for adapter capacity. And the discipline: watch **eval loss**, not train loss. Train loss always looks great right up until the model is ruined.

```
TrainingArguments(learning_rate=2e-4, num_train_epochs=2)
```

Naved Khan
Senior Gen-AI Engineer

Q13 Where does good SFT data come from?

- **Production**, ideally: real conversations where the user accepted the answer, human-reviewed and PII-scrubbed. That's the data flywheel.
- It beats hand-written examples because it carries the **real distribution**: actual phrasing, actual mess, actual edge cases.

Q14 Synthetic training data: smart or dangerous?

- Both, same as with evals. A strong model can **generate and augment** examples cheaply, great for coverage of rare cases.
- The risks: **style collapse** (the student sounds like the generator, not your users) and error amplification. Human review on a sample is non-negotiable.

Naved Khan
Senior Gen-AI Engineer

Q15 What is data contamination in this context?

- Eval examples **leaking into training data**. Your scores look brilliant because the model memorized the test.
- Exact-match dedupe isn't enough: near-duplicates leak too. Dedupe train against eval by **similarity**, and treat any suspiciously perfect score as a bug until proven otherwise.

Q16 Why does the chat template matter so much?

- Models are trained on conversations wrapped in a **specific token format**. Train with one template, serve with another, and quality silently degrades.
- It's the most boring bug in fine-tuning and one of the most common. The fix is one rule: **identical template in training and inference**, verified, not assumed.

Naved Khan
Senior Gen-AI Engineer

Q17 How do you know if you have enough data?

- **Learning curves**: train on 25%, 50%, 100% and plot eval scores. Still climbing at 100%? More data helps.
- Flat curve? More of the same won't help; shift to **quality and diversity**. Ten minutes of plotting replaces weeks of guessing.

Q18 Your task has rare but critical edge cases. Data strategy?

- **Oversample the rare intents** so the model actually sees them, and consider synthetic augmentation for the truly scarce ones.
- Then evaluate **per segment**: an aggregate score hides exactly the failures you oversampled to fix.

Naved Khan
Senior Gen-AI Engineer

Q19 How do you evaluate a fine-tune?

- Three layers: a **held-out test set** for the task, the **same golden suite production runs**, and a general-capability check for forgetting.
- And it ships like any change: **behind the eval gate, canaried**, rollbackable. A fine-tune is a deploy, not an experiment that goes straight to users.

Q20 What baseline must every fine-tune beat?

- **The best prompt on the base model.** Not the lazy prompt, the best one. Many fine-tunes lose that comparison, and teams only discover it after spending the GPU money.
- Saying this unprompted is one of the strongest senior signals in the whole topic.

```
assert ft_score > best_prompt_score  # else don't ship it
```

Naved Khan
Senior Gen-AI Engineer

Q21 How do you spot overfitting in a fine-tune?

- **Eval loss rising while train loss falls**: the curves tell you first. Then behavioral tells: verbatim phrases from training data, and rigidity on anything slightly out of distribution.
- The fix is usually less: **fewer epochs, lower rate, more diverse data**. With LLMs, undertraining is the safer error.

Q22 What does a LoRA run actually cost?

- Order of magnitude, which is what interviewers want: a 7–8B model with QLoRA is **hours on a single GPU**, and you're training well under 1% of the parameters.
- The real cost isn't compute, it's the **data work and the eval work** around the run. Budget accordingly and say so.

Naved Khan
Senior Gen-AI Engineer

Q23 How do you version and roll back fine-tunes?

- The **adapter is the artifact**: versioned together with its dataset snapshot, training config, and eval report. Reproducible or it didn't happen.
- Rollback is instant because the base never changed: **swap the adapter back**. One more reason PEFT won.

Q24 How do you serve fine-tuned models?

- Two shapes: **merge** the adapter into the weights for one fast dedicated model, or **multi-LoRA serving**, many adapters hot-swapped on one shared base.
- Multi-LoRA is the answer for per-tenant or per-task customization: hundreds of fine-tunes, one GPU pool. Provider fine-tuning APIs trade this control for convenience.

Naved Khan
Senior Gen-AI Engineer

Q25 How does fine-tuning connect to the cost story?

- It's the **deepest cost lever**: distill routine traffic into a small fine-tuned model and serve it 10× cheaper than the frontier model it replaces.
- But it comes **last**, after caching and routing, because it's the least reversible. And it ships behind the same eval gate as everything else.

Q26 Provider fine-tuning API or open weights?

- **API**: fastest path, zero infra, but your adapter lives in their cloud and moves at their roadmap's pace.
- **Open weights**: ownership, portability, data governance, multi-LoRA serving, at the price of running infra. Regulated industries usually end up open-weights; startups usually start API.

Naved Khan
Senior Gen-AI Engineer

Q27 Fine-tuning or RAG: how do you end the debate?

- Refuse the either-or. The production pattern is **both**: RAG supplies the facts, fine-tuning shapes the behavior, including **how the model uses retrieval**: citing properly, abstaining when evidence is thin.
- One sentence: **knowledge changes daily, behavior changes quarterly**. Match the tool to the rate of change.

Q28 The most common fine-tuning failure you've seen?

- **Fine-tuning a knowledge problem.** The facts change, the model is stale, and now there's a retraining treadmill that RAG would have avoided entirely.
- Second place: no baseline and no evals, which produces the classic "we fine-tuned and it got worse, we think." If you can't measure it, you fine-tuned blind.

Naved Khan
Senior Gen-AI Engineer

Q29 Is fine-tuning growing or dying in 2026?

- Casual fine-tuning **shrank**: better base models, cheap frontier APIs, and prompting plus RAG cover most needs.
- But two uses are **growing fast**: distillation for cost, and domain-specific small models at the edge. Less fine-tuning overall, more of it done for the right reasons.

Q30 So: "Should we fine-tune?" Give the senior answer.

- A decision tree, spoken calmly: **What's the gap: knowledge, behavior, cost, or latency?** Knowledge → RAG. Behavior → prompt hard first. Cost or latency → cascade and cache first.
- Only if the gap survives all that, and we have clean data plus an eval suite to prove the win: **then yes, LoRA first, and it ships like any other deploy**.

Naved Khan
Senior Gen-AI Engineer

Preparing for AI interviews?

Naved Khan

Senior Gen-AI Engineer

I post practical GenAI interview prep, **RAG, agents, and production AI**, every week. Follow along if that's useful to you.